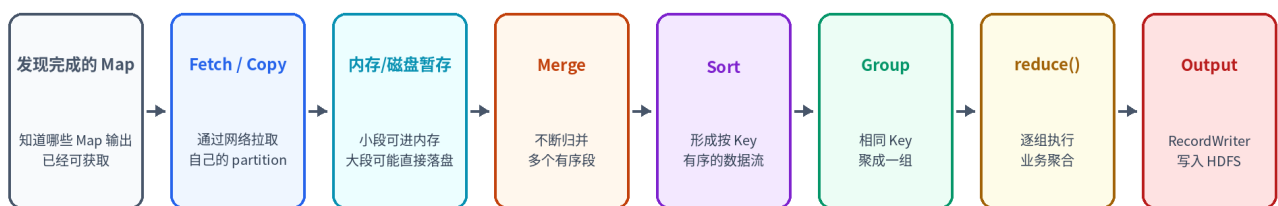


Hadoop 中的 MapReduce 原理详解

第四讲：Shuffle 与 Reduce 端执行机制

网络 Fetch · Merge Sort · Grouping · reduce() · HDFS 输出

Shuffle 到 Reduce：Reducer 端真正经历的流水线



官方 Reducer API 把 Reducer 的主要阶段概括为：Shuffle -> Sort -> Reduce。实际执行中 Shuffle 与 Sort/merge 会交叠发生。

理解重点：Shuffle 不是“把所有 Map 输出复制给所有 Reducer”，而是每个 Reducer 只从每个 Mapper 取属于自己的那一个 partition 段。

最后交给 reduce(key, Iterable<values>) 的，已经不是零散记录，而是“同一个 key 对应的一整组 value”。

目标：真正看懂 MapReduce 中最昂贵、最关键的“跨节点数据重组”阶段

学习目标

第三讲停在一个关键位置：每个 Mapper 已经把自己的中间结果整理成“按 Reducer 分区、分区内按 Key 有序”的本地输出。现在问题来了：Reducer 怎样把散落在很多机器上的属于自己的那一段数据拿回来？拿回来以后怎样在有限内存下继续归并？又怎样把相同 Key 变成一组 Value，最终调用 reduce()？这一讲把这条链完整打通。

- 区分狭义 Shuffle 与广义 Shuffle，知道不同教材为什么表述不完全一致。
- 理解 Reducer 只拉取每个 Mapper 中属于自己的 partition，而不是复制整个 Map 输出。
- 理解 Reducer 为什么需要并行 Fetch、内存缓冲、磁盘段和多路 Merge。
- 理解 Shuffle 与 Sort 为什么是交叠进行的，而不是完全串行的两个阶段。
- 理解 Sort 与 Grouping 的区别，以及 reduce(key, Iterable<values>) 的输入如何形成。
- 理解 Reducer 可以在部分 Mapper 完成后开始 Shuffle，但必须等所需 Map 输出全部到齐后才能完成该阶段。
- 理解 reduce() 的 setup / reduce / cleanup 生命周期和最终输出过程。
- 从性能视角理解 Shuffle 为什么常是 Hadoop MapReduce 的主要成本中心。

本讲总链条

每个 Mapper 的目标 partition -> Reducer 通过网络 Fetch -> 内存/本地磁盘保存多个有序段 -> 边拉边 Merge -> 全部到齐后形成最终有序输入 -> Group 相同 Key -> reduce(key, values) -> RecordWriter -> HDFS。

0. 先把“Shuffle”这个词说清楚

0.1 狭义 Shuffle：网络搬运

从字面和官方 Reducer API 的阶段划分看，Shuffle 主要指 Reducer 从各个 Mapper 所在节点，通过网络复制自己需要的已排序 Map 输出分区。这个意义上，Shuffle 的核心动作就是 Copy / Fetch。

0.2 广义 Shuffle：跨 Map 与 Reduce 的整段重组过程

很多教材和工程讨论会把 Partition、Map 端排序、网络 Fetch、Reduce 端 Merge/Sort、Grouping 等统称为 Shuffle。原因是这些机制共同完成一件事：把“按输入 Split 分散的数据”重新按照 Key 组织，让同一个 Key 的所有记录在同一个 Reducer 中相遇。

说法	通常包含什么	适合什么时候用
狭义 Shuffle	Reducer 通过网络 Fetch Map 输出分区	解释 Reducer 官方阶段、网络流量时
广义 Shuffle	Partition + Sort + 网络传输 + Merge + Group	解释“按 Key 重分布”的整体机制时

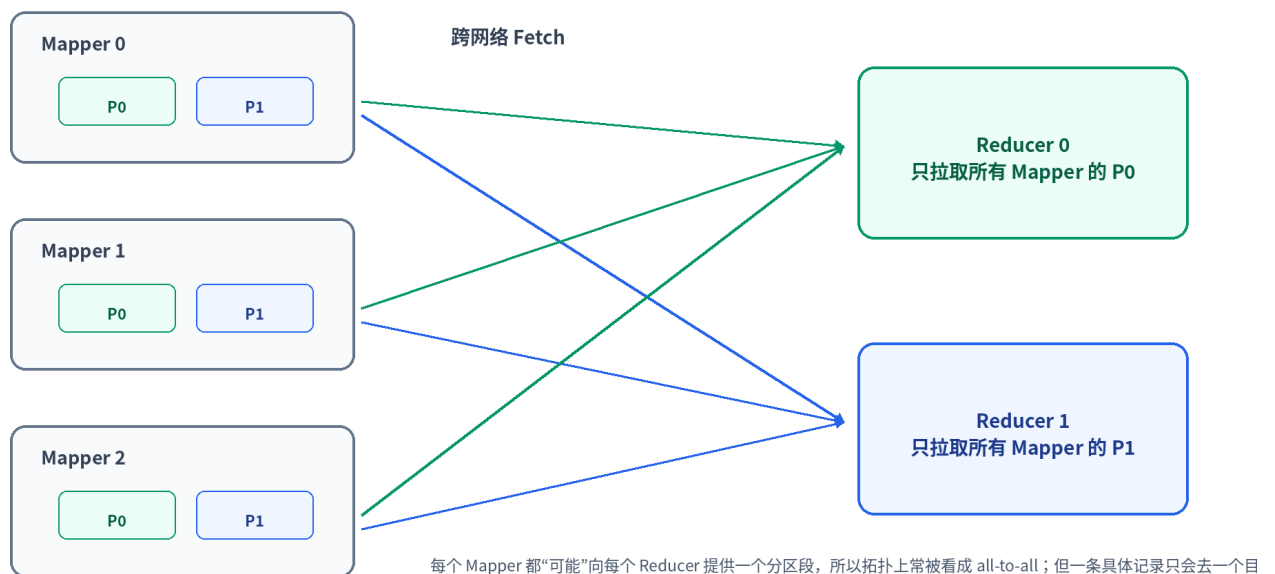
本课程约定

后面说“Shuffle 阶段”时，我会明确当前是在说“网络 Fetch”还是“广义数据重组”。真正重要的不是争术语，而是知道每一步的数据在哪里、为什么要做。

1. 从第三讲接起来：每个 Mapper 手里已经有什么？

假设 Job 有 3 个 Mapper、2 个 Reducer。每个 Mapper 完成后，都留下一个本地中间输出，其中逻辑上包含 P0 和 P1 两个分区段；每个分区内部的 Key 已经有序。P0 属于 Reducer 0，P1 属于 Reducer 1。

3 个 Mapper + 2 个 Reducer：Shuffle 的网络关系到底是什么



因此 Reducer 0 的任务非常明确：去所有 Mapper 的输出中把 P0 拉回来。Reducer 1 同理只拉 P1。一个 Mapper 的输出会被多个 Reducer “按段读取”，但一条具体记录只属于一个 partition，也只进入一个 Reducer。

一个容易混淆的说法

Shuffle 的通信拓扑常被称为 all-to-all，因为每个 Mapper 都可能给每个 Reducer 提供一个分区段；但这不等于“一条记录广播给所有 Reducer”。记录仍然只被 Partitioner 指定给一个 Reducer。

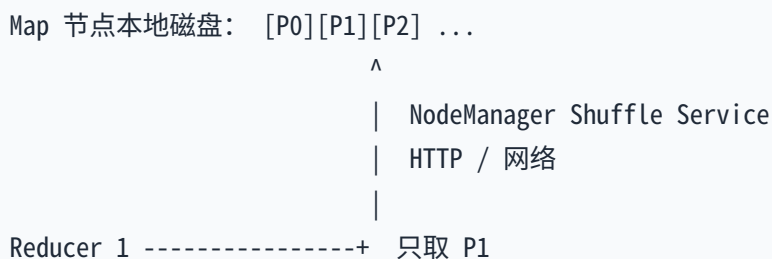
2. Reducer 到底从哪里拉数据？

2.1 Map 输出不是放在 HDFS，而是在 Map 节点本地

第三讲已经讲过，经典 MapReduce 中间输出通常存放在运行 MapTask 的节点本地磁盘。这样做避免了中间数据写 HDFS 时的复制与持久化成本，但也带来一个要求：Reducer 必须知道目标 Map 输出在哪个节点，并通过网络去获取。

2.2 YARN 中由 NodeManager 的 Shuffle 服务配合提供数据

在 YARN 模式下，节点侧通常配置 mapreduce_shuffle 辅助服务。Reducer 作为 Shuffle 消费者，通过网络向保存 Map 输出的节点发起请求。Apache Hadoop 官方 Reducer API 明确描述：Reducer 的 Shuffle 阶段会通过 HTTP 复制各 Mapper 的已排序输出。



这里的核心不是记住 HTTP 这个协议细节，而是理解“计算节点之间直接交换中间数据”。HDFS 负责持久数据，Shuffle 服务负责把临时 Map 输出高效送到目标 Reducer。

3. Reducer 什么时候开始？必须等所有 Mapper 吗？

3.1 不需要等全部 Mapper 都结束才启动 ReduceTask

为了让网络传输与 Map 计算重叠，MapReduce 可以在一部分 Mapper 完成后就调度 Reducer。Reducer 一旦运行，就可以开始 Fetch 那些已经完成的 Mapper 输出。是否以及何时启动 Reducer，会受到 reduce slow-start 相关配置、可用 YARN 资源和调度器状态等因素影响。

Reduce 不是等所有 Mapper 全结束才“突然开始”：调度、Fetch、Merge 可以与剩余 Map 运行重叠

3.2 但是 reduce() 不能凭空少算尚未完成的 Mapper

Reducer 可以提前“搬货”和“整理货物”，但在一个 Key 的全局输入还没确定之前，不能把缺少后续 Mapper 贡献的数据当作最终组来计算。因此 Shuffle/Sort 可以和 Map 阶段重叠，而真正完成所需数据汇集后才进入正式的 Reduce 处理。

系统设计思想

这是一种典型流水线化：能提前做的网络 I/O 与归并就提前做，减少“所有 Mapper 结束后全体 Reducer 才从零开始”的空等。

4. Fetch：Reducer 如何同时从很多 Mapper 拉数据？

4.1 不是一个 Mapper 一个 Mapper 串行下载

如果有 1000 个 Mapper，Reducer 一个接一个串行 Fetch 会非常慢。实际框架支持并行复制多个 Map 输出。参数 `mapreduce.reduce.shuffle.parallelcopies` 就是用来控制 Reduce 端并行 Copy 数量的一个重要配置项。集群管理员可以根据网络、节点规模和负载调整它。

4.2 每次 Fetch 的对象是“这个 Reducer 对应的分区段”

Reducer 2 需要：

Map 0 -> P2

Map 1 -> P2

Map 2 -> P2

...

Map 999 -> P2

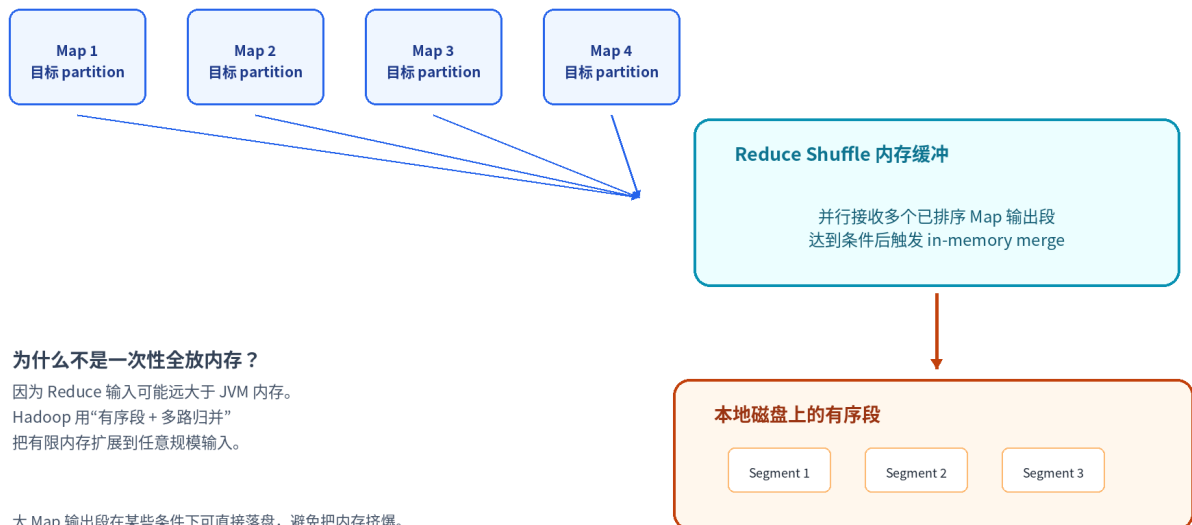
而不是：

Map 0 的完整 [P0 P1 P2 ...] 全部下载

这正是 Map 端提前 Partition 的价值：Reducer 无需下载无关数据再自己筛选。网络流量从一开始就只传“属于这个 Reducer 的那一段”。

5. Fetch 回来的数据放哪里？内存不够怎么办？

Reducer 拉取数据后：为什么还要“边拉、边合并、必要时落盘”



5.1 小的 Map 输出段可以先进入 Reduce Shuffle 内存

Reduce 端会分配一部分 JVM 堆内存用于暂存 Fetch 回来的 Map 输出。相关参数包括 `mapreduce.reduce.shuffle.input.buffer.percent` 等。这样可以减少每个小段都立即写磁盘造成的 I/O。

5.2 内存是有上限的，所以必须不断 Merge / Spill 到磁盘

当已获取的 Map 输出越来越多，内存达到合并触发条件时，框架会把若干有序段归并后写到本地磁盘。官方教程还指出：过大的单个 Map 输出段可能直接写到磁盘，而不是先完整经过内存。

5.3 为什么“有序段”让有限内存可以处理超大 Reduce 输入？

每个 Map 输出分区本来就已经按 Key 排好序，因此 Reducer 不需要把所有记录一次性加载成一个巨型数组再排序。它可以维护多个有序输入流，用多路归并逐步得到更大的有序流。这个技术和外部排序（External Sort）的思想完全一致。

底层本质

内存不够时，不是“算法失败”，而是把大问题分成多个已排序的磁盘段，再做多路归并。MapReduce 能处理远大于单机内存的数据，外部排序思想是关键原因之一。

6. Shuffle 和 Sort 为什么说是“同时发生”的？

官方 Reducer 文档明确指出：Shuffle 和 Sort 阶段是同时发生的，Map 输出一边被 Fetch，一边被 Merge。这里的“Sort”不要理解成“等网络全部完成后从头快速排序一次”，而应该理解为持续维护和归并多个已经有序的 Map 输出段。

```
时间 t1: Fetch 到 Map0/P0 -> 有序段 A
时间 t2: Fetch 到 Map3/P0 -> 有序段 B
时间 t3: A + B 达到条件 -> Merge 成有序段 AB
时间 t4: 又 Fetch C、D -> 继续归并
...
最后: 所有目标 Map 输出到齐 -> 得到完整的有序 Reduce 输入
```

这种“边获取边归并”有两个直接收益：第一，网络和磁盘/CPU 工作可以重叠；第二，不必让成百上千个小段一直堆到最后才处理。

7. Merge Sort：为什么 Reducer 不需要重新做昂贵的全量排序？

7.1 前提：每个 Mapper 提供给某个 Reducer 的分区本来就是有序的

第三讲已经说明，Map 端 Spill 和最终 Merge 会让每个 partition 内部按 Key 有序。假设 Reducer 0 收到三个 Mapper 的分区：

```
Map A: apple, cat, dog
Map B: apple, banana, cat
Map C: banana, cat, zebra
```

Reducer 可以用多路归并不断比较各有序流当前最小 Key，按顺序取出：apple, apple, banana, banana, cat, cat, cat, dog, zebra。这个过程的核心是“合并已有序序列”，而不是把所有元素打乱后再从零排序。

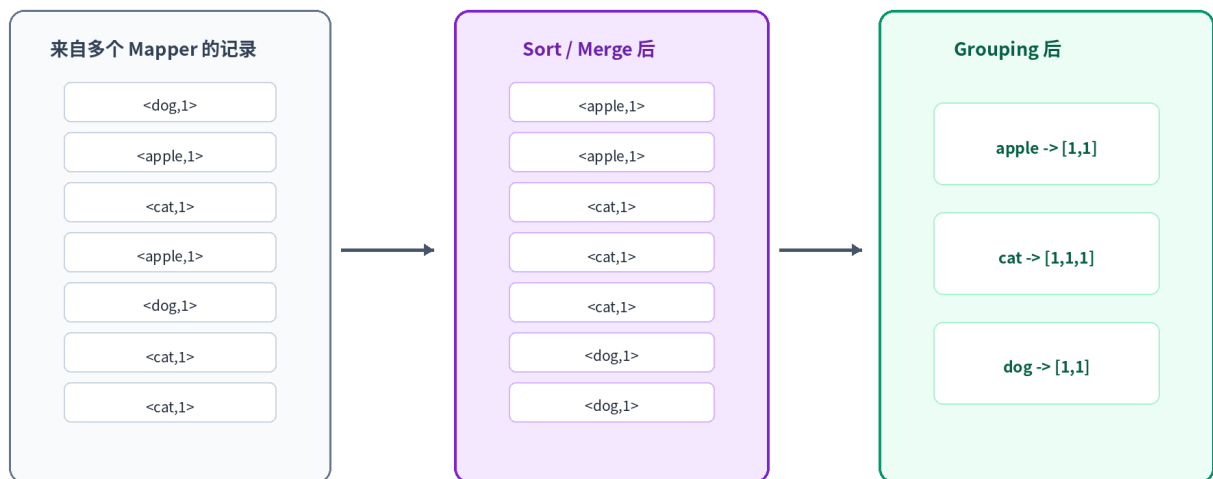
7.2 磁盘上的段太多时还要分多轮归并

文件描述符、磁盘读写和压缩器数量都有现实限制，所以一次不能无限打开所有段。

mapreduce.task.io.sort.factor 等参数会影响一次归并多少段。段很多时会出现多轮 merge pass：先把一批小段归成中段，再把中段继续归并。

8. Sort 与 Grouping: 为什么 Reducer 收到的是 <key, Iterable<values>>?

Sort 与 Group 是两件事：先让 Key 有序，再决定哪些 Key 属于“同一组”



默认情况下“排序比较规则”和“分组比较规则”通常都以 Key 为依据；高级的 Secondary Sort 会把二者分开控制。

8.1 Sort 解决“顺序”，Grouping 解决“边界”

排序以后，相同 Key 会相邻出现。但 reduce() 不是每见到一条 <cat,1> 就调用一次，而是要知道“cat 这一组从哪里开始、在哪里结束”。Grouping Comparator 就是在有序流上定义哪些 Key 被视为同一组。默认情况下，相等 Key 会归为一组。

有序输入：

```
apple 1
apple 1
cat 1
cat 1
cat 1
dog 1
```

分组后：

```
apple -> Iterable(1,1)
cat -> Iterable(1,1,1)
dog -> Iterable(1)
```

8.2 Iterable<values> 不等于 “一定先创建一个巨大 List”

Java 接口写的是 `Iterable<VALUEIN>`，核心意思是“Reducer 可以逐个遍历这组 Value”。对超大 Key 来说，把几千万个 Value 全部复制进一个 Java List 既耗内存又没必要。框架利用有序输入，能够按组边迭代边消费。因此写 Reducer 时最好用单遍迭代思维，不要假设 values 可以随意重复遍历或永久持有其中的可变对象。

迁移到算法思维

很多大数据框架都会把“全量随机访问”改造成“有序流 + 单遍迭代”，用访问约束换取可扩展性。理解这一点，比死记 Hadoop 类名更重要。

9. Sort Comparator 与 Grouping Comparator 为什么可以不同？

普通 WordCount 中，排序和分组都只看 word，所以你感觉它们是一件事。但高级场景里，可以把 Key 设计成复合结构，例如 `<department, salary>`。排序时按 department 再按 salary 排，而分组时只按 department 分组。这样 Reducer 收到同一部门的一组记录时，组内已经按 salary 有序，这就是 Secondary Sort（二次排序）的基本思想。

复合 Key: <dept, salary>

Sort Comparator: 先 dept, 再 salary

Grouping Comparator: 只比较 dept

结果:

IT, 5000

IT, 7000

IT, 9000 -> 同一次 reduce(IT, values), 但 values 对应记录已按 salary 顺序到达

当前先记住

Sort 决定“全体中间 Key 的顺序”；Grouping 决定“哪些连续 Key 被视作同一次 reduce() 的一组”。后面需要 Top-N、分组内排序时再深入二次排序。

10. reduce() 生命周期：Reducer 业务代码到底什么时候运行？

10.1 setup(): 每个 ReduceTask 开始时一次

Reducer 类可以覆盖 setup(Context)，用于加载配置、初始化连接或创建只需一次的对象。它不是每个 Key 调用一次，而是该 ReduceTask 生命周期开始时调用一次。

10.2 reduce(key, values): 每个分组调用一次

```
protected void reduce(Text key, Iterable<IntWritable> values, Context context) {
    int sum = 0;
    for (IntWritable v : values) {
        sum += v.get();
    }
    context.write(key, new IntWritable(sum));
}
```

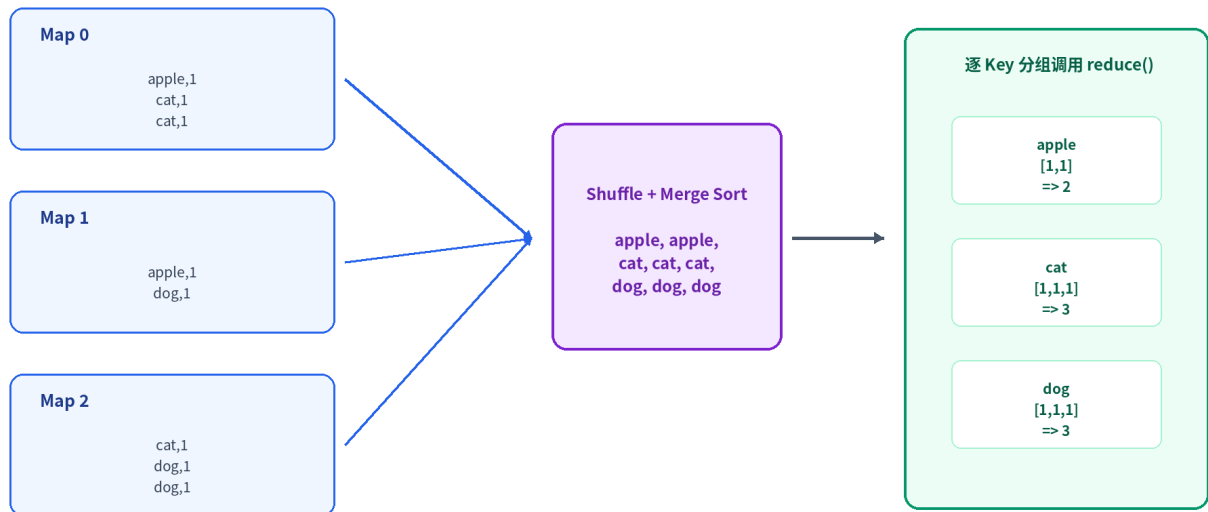
如果当前 Reducer 负责 100 万个不同 Key，那么 reduce() 大约会被调用 100 万次；每次对应当前 Key 的一整组 Value。

10.3 cleanup(): 整个 ReduceTask 结束时一次

cleanup(Context) 用于释放资源、关闭自定义连接等。标准输出写入一般由 context.write() 交给框架的 RecordWriter 处理。

11. WordCount: 把整个 Shuffle + Reduce 走一遍

WordCount : 从 3 个 Mapper 的局部有序段, 到 Reducer 的 <key, values>



最终输出 (示意) : apple 2 cat 3 dog 3

注意: Iterable<values> 是逻辑上的“这一组所有值”。不要把它简单理解成框架一定先构造一个巨大 Java List; 框架更倾向于基于有序输入迭代地提供这一组数据。

1. Map 0、Map 1、Map 2 分别产生局部中间结果, 并在 Map 端整理成有序分区。
2. 目标 Reducer 从三个 Mapper 所在节点拉取属于自己的 partition。
3. 三个来源本身各自有序, Reducer 边 Fetch 边归并。
4. 最终有序流变成 apple, apple, cat, cat, cat, dog, dog, dog。
5. Grouping 识别 apple 组、cat 组、dog 组。
6. 框架调用 reduce("apple", [1,1]), 得到 2; 再调用 cat 组, 得到 3; 再调用 dog 组, 得到 3。
7. context.write() 把最终记录交给 OutputFormat / RecordWriter, 写入 HDFS。

真正的 MapReduce 本质

Map 阶段是“按输入位置并行”; Shuffle 把数据从“按输入位置组织”转换成“按业务 Key 组织”; Reduce 才能在这个新组织方式上做全局聚合。

12. Reducer 输出：为什么会看到 part-r-00000?

12.1 一个 Reducer 通常对应一个结果 part 文件

FileOutputFormat 场景下，Reducer 最终通过 RecordWriter 把结果写入任务输出。一个成功的 Reducer 通常形成一个 part-r-xxxxx 文件。例如只有一个 Reducer 时，WordCount 常见输出就是 part-r-00000；有 4 个 Reducer 时通常会看到 part-r-00000 到 part-r-00003。

```
output/
|_ _SUCCESS
|_ part-r-00000
|_ part-r-00001
|_ part-r-00002
|_ part-r-00003
```

12.2 多个 Reducer 的输出不是“自动全局有序”

每个 Reducer 的输入是按 Key 有序的，但 Reducer 输出并不会被框架再次自动排序。更重要的是，默认 HashPartitioner 只是按 hash 分区，不保证 Reducer 0 的 Key 全都小于 Reducer 1 的 Key。因此多个 part-r 文件不能简单视作一个天然全局有序文件。需要全局顺序时，要设计范围分区、TotalOrderPartitioner 等方案，或使用单 Reducer（但这会牺牲并行度）。

13. 为什么 Shuffle 往往是 MapReduce 最贵的一段？

成本来源	发生了什么	为什么昂贵
网络 I/O	每个 Reducer 从许多 Map 节点拉取分区	集群带宽有限，跨机传输远慢于内存访问
磁盘 I/O	Map 端 Spill/Merge，Reduce 端也可能多次落盘/归并	大数据远超内存，外部排序不可避免
CPU	序列化、反序列化、排序、压缩/解压	记录数巨大时累计成本高
同步/等待	Reducer 最终依赖所有相关 Map 输出	慢 Map、失败重试会形成长尾
数据倾斜	某些 partition 或热点 Key 特别大	一个 Reducer 拖住整个 Job
小段/连接管理	大量 Mapper 产生大量待 Fetch 段	连接、索引、merge pass 和调度开销上升

这也是为什么很多 MapReduce 优化建议最终都在围绕同一件事：尽量减少 Shuffle 的“数据量、碎片数、不均衡程度和磁盘往返次数”。

14. 性能优化：先看因果链，不要背参数

问题	优先考虑的机制	原因
Map 中间结果太大	Combiner、减少无用字段、过滤、Map 输出压缩	直接减少磁盘和网络字节数
某个 Reducer 特别慢	检查热点 Key / partition 倾斜	单纯加机器可能救不了热点分区
Fetch 很慢	网络、并发 Copy、Map 输出压缩	Shuffle 本质是跨节点数据搬运
Reduce 端 merge 很多	检查段数、缓冲与 merge 参数	过多小段带来多轮磁盘归并
Reducer 太少	适当增加 Reduce 并行度	单个 Reducer 输入过大
Reducer 太多	减少 Reducer 或控制小文件	Task/连接/最终 part 文件数量开销上升

14.1 几个需要“认识”的 Reduce/Shuffle 参数

配置项	作用	不要怎么理解
mapreduce.reduce.shuffle.parallelcopies	Reduce 端并行 Fetch 数量	不是越大越好，受网络/节点负载限制
mapreduce.reduce.shuffle.input.buffer.percent	Shuffle 可用于保存 Map 输出的内存比例	不是给 reduce() 业务代码的全部内存
mapreduce.reduce.shuffle.merge.percent	触发内存中 Map 输出合并的阈值之一	是 merge 频率权衡，不是正确性参数
mapreduce.reduce.input.buffer.percent	正式 Reduce 开始时可保留 Map 输出在内存中的比例	默认策略与集群配置可调整
mapreduce.task.io.sort.factor	一次 merge 可涉及的流/段数量上限因素之一	太高也会增加资源占用

优化顺序

先减少不必要的数​​据，再解决倾斜，再看并行度和压缩，最后才是细调 buffer/merge 参数。因为“传 1 TB 数据”通常比“把 1 TB 数据的 buffer 参数调漂亮”更根本。

15. 容错：Shuffle 中途某个节点出问题怎么办？

15.1 Reducer Fetch 失败不等于 Job 必然失败

网络抖动、节点临时不可达或 Map 输出文件丢失都可能导致 Fetch Failure。框架会重试获取；如果确认某个 Map 输出已不可用，可以重新执行相应 MapTask，重新生成中间输出。

15.2 为什么 Map 中间结果敢只放本地？因为它可重算

这是前几讲反复出现的设计：输入数据可靠保存在 HDFS，Map 函数在同样输入下可以重新执行，因此 Map 中间输出不需要像最终数据那样做昂贵的 HDFS 多副本持久化。牺牲少量失败后的重算时间，换取正常情况下更高吞吐。

一个普适思想

“持久化所有中间状态”并不是唯一的可靠性方案；如果上游输入可靠、计算可重放，那么“失败后重算”常比“每步都强持久化”更便宜。Spark 的 lineage、现代数据流水线也大量使用类似思想。

16. 初学者最常见的 16 个疑问

Q1: Shuffle 是不是 Reduce 函数的一部分？

不是业务 reduce() 本身。它是框架为了准备 Reducer 输入而执行的数据搬运、归并、排序等工作。

Q2: Reducer 会把每个 Mapper 的完整输出都下载一遍吗？

不会。它只 Fetch 自己对应的 partition 段。

Q3: 一个 Mapper 会不会把同一条记录发给两个 Reducer？

默认不会。一条 Map 输出记录通过 Partitioner 得到唯一 partitionId。

Q4: 为什么说 Shuffle 是 all-to-all？

拓扑上每个 Mapper 都可能为每个 Reducer 提供一个分区段；但具体记录不是广播。

Q5: Reducer 必须等所有 Mapper 完成才启动吗？

不一定。可以提前被调度并 Fetch 已完成 Mapper 的输出；具体受 slow-start、资源和调度影响。

Q6: Reducer 能不能在只拿到 80% Map 输出时就算最终结果？

不能把缺少剩余 Map 贡献的数据当最终全局输入。它可以提前 Fetch/merge，但最终组必须完整。

Q7: 为什么 Shuffle 和 Sort 会同时发生？

因为每拿到一些已排序 Map 段就可以开始 merge，不必等所有网络传输结束。

Q8: Reducer 还要“重新排序”吗？

更准确说是对多个已经有序的 Map 段做 merge sort，形成完整有序输入。

Q9: Grouping 和 Sort 是同一件事吗？

不是。Sort 决定顺序；Grouping 决定哪些相邻 Key 被送入同一次 reduce()。

Q10: values 是 Java List 吗？

接口是 Iterable。不要依赖它一定是可随机访问、可重复遍历的 List。

Q11: 一个热点 Key 太大，增加 Reducer 数量就能解决吗？

不一定。同一 Key 必须去一个 Reducer，单热点 Key 仍可能造成倾斜，需要业务级拆分或两阶段聚合。

Q12: Map 输出压缩会不会让网络更快？

常常能减少网络和磁盘字节，但会增加压缩/解压 CPU，需根据数据和 codec 权衡。

Q13: 多个 Reducer 的 part 文件拼起来就是全局排序吗？

默认 HashPartitioner 下不是。每个文件内部的输出也不由框架再次自动排序。

Q14: 为什么不用 HDFS 做 Shuffle 中转？

那会让所有临时数据承担持久化和副本成本，增加网络与 NameNode 压力。

Q15: Reducer 失败后要重新跑所有 Mapper 吗？

通常不需要；新的 Reduce attempt 可以重新 Fetch 已成功 Map 的输出。只有缺失的 Map 输出必要时才需重算。

Q16: Shuffle 是不是 MapReduce 慢于某些内存计算框架的主要原因？

是重要原因之一。MapReduce 的外部排序、磁盘中间结果和网络重分区带来较高 I/O 成本，尤其迭代算法会反复支付这些成本。

17. 迁移练习：统计每个用户访问网站的总次数

任务：100 GB Web 日志中，每行包含 userId、URL、timestamp。Mapper 输出 <userId,1>，配置 6 个 Reducer。请从 Shuffle 视角回答。

问题 1: Reducer 3 从每个 Mapper 拉什么？

参考答案

只拉 partition 3，而不是整个 Mapper 输出。

问题 2: 为什么 Mapper 要提前把 partition 3 内的 userId 排序？

参考答案

这样 Reducer 3 从很多 Mapper 得到的都是有序段，可以多路归并并快速识别同 userId 的组边界。

问题 3: 如果 user_001 占全部日志 30%，为什么会慢？

参考答案

所有 user_001 必须进同一个 Reducer，形成单 Key 热点，导致该 Reducer 输入和 reduce() 工作量远大于其他 Reducer。

问题 4: Combiner 在这个任务中有什么价值？

参考答案

Mapper 本地先把同 userId 的多个 1 合成 localCount，直接减少 Shuffle 记录数。

问题 5: 如果 Reducer 还没等最后一个 Mapper 完成，能做什么？

参考答案

可以继续 Fetch 已完成 Map 输出并 merge；但不能宣告该 userId 的最终总数完成。

问题 6：输出为什么可能有 6 个 part-r 文件？**参考答案**

6 个 Reducer 各自写自己的最终输出分片。

18. 本讲最终心智模型

层次	Reducer 在做什么	你应该抓住的本质
Fetch	从所有 Mapper 取自己的 partition	跨节点按 Key 重分布
Buffer	用内存暂存一部分 Map 输出	减少细碎磁盘 I/O
Merge	把多个有序段不断合并	外部排序让数据规模突破内存限制
Sort	形成完整按 Key 有序流	相同 Key 靠在一起
Group	识别同 Key 的连续区间	形成 <key, Iterable<values>>
Reduce	对每个 Key 组执行聚合	真正业务逻辑发生处
Output	RecordWriter 写 HDFS	生成可持久使用的结果

如果只记一句话，可以记成：Shuffle 的本质，是把“分散在各 Mapper、按输入位置组织的数据”，跨网络重新组织成“在 Reducer 上按业务 Key 聚集的数据”；Sort/Merge 让这种重组在有限内存下可扩展，Grouping 再把有序流切成一个个可以交给 reduce() 的 Key 组。

核心因果链

Partition 决定去哪 -> Fetch 把目标分区搬来 -> Merge/Sort 把很多有序段汇成一个有序流 -> Group 找到同 Key 的边界 -> reduce() 才能做完整聚合。

附录：本讲术语速查

术语	通俗解释
Shuffle	狭义指 Reducer 网络拉取 Map 输出；广义常指按 Key 跨节点重组的整段过程。
Fetch / Copy	Reducer 从 Map 节点获取自己对应 partition 的网络复制。
Shuffle Service	节点侧为 Reducer 提供 Map 输出的服务，YARN 常配置 mapreduce_shuffle。
Segment	一段已经按 Key 有序的中间数据，可位于内存或磁盘。

术语	通俗解释
Merge Sort	把多个已排序段合成一个更大的有序流。
Grouping	根据比较规则把连续的相同/等价 Key 划为一组。
Sort Comparator	决定中间 Key 的排序顺序。
Grouping Comparator	决定哪些 Key 被视为同一 reduce 组。
Secondary Sort	通过复合 Key + 不同排序/分组规则实现“组内有序”。
Reduce Slow Start	控制何时开始调度 ReduceTask 的一类机制/配置。
Fetch Failure	Reducer 获取 Map 输出失败，框架可重试，必要时重跑 Map。
part-r-xxxxx	Reducer 常见的最终输出分片文件名。

下一讲预告：MapReduce 与 YARN 的完整 Job 生命周期

前四讲已经把“数据怎样流动”讲通。下一讲建议把视角从数据流提升到“谁在调度这些任务”：Client 提交 Job 后，ResourceManager、MRAppMaster、NodeManager、Container 分别做什么；MapTask 和 ReduceTask 怎样申请资源、启动、汇报状态、失败重试；最终你在命令行执行 `hadoop jar` 后，集群内部到底发生了哪些事件。

参考依据（Apache Hadoop 官方文档）

- 1) Apache Hadoop 3.3.5 MapReduce Tutorial：Reducer 的 Shuffle/Sort/Reduce 阶段、Shuffle/Reduce 内存与 merge 参数、WordCount 示例。
- 2) Apache Hadoop 3.3.5 Reducer API：Reducer 的三个主要阶段；Shuffle 通过网络复制 Mapper 已排序输出，Shuffle 与 Sort 交叠。
- 3) Apache Hadoop 3.3.5 Cluster Setup / Single Node Setup：YARN 的 `mapreduce_shuffle` 辅助服务及 MapReduce 运行配置。
- 4) Apache Hadoop 3.3.5 Pluggable Shuffle and Pluggable Sort：默认 ShuffleHandler / Shuffle 等组件背景。

版本说明

本教程以 Hadoop 3.3.5 的经典 MapReduce 行为为基线。具体阈值与默认参数可能被集群配置覆盖；学习时优先掌握“为什么要 Fetch、Merge、Sort、Group”这些不随调参变化的机制。